

TaskAPI
Sistema de Gestión de Tareas

Integrantes:

Marbin Ronaldo Meneses

Brayan Bambague Gallardo

Yordi

Jhonathan

Presentado a:

Leidy Mariani Dorado Flórez

Corporación Universitaria Comfacauc

Popayán - 2025 P1

Descripción del problema

Actualmente muchas personas gestionan sus tareas diarias de forma manual o utilizan aplicaciones que no generan recordatorios automáticos personalizados. Esto puede ocasionar que las tareas se olviden o no se realicen a tiempo.

El sistema TaskAPI busca solucionar este problema permitiendo a los usuarios registrar tareas y recibir notificaciones automáticas en Telegram cuando estas se encuentran pendientes.

De esta manera el usuario puede mantener un mejor control sobre sus actividades personales o laborales.

Arquitectura implementada

El sistema se implementa como una arquitectura distribuida basada en microservicios, en la cual cada funcionalidad principal se desarrolla como un servicio independiente.

Los servicios que componen el sistema son:

- Servicio de usuarios: encargado del registro, autenticación y gestión de usuarios.
- Servicio de tareas: encargado de la creación, consulta y actualización de tareas.
- Servicio de notificaciones: encargado de consultar tareas pendientes y enviar recordatorios a los usuarios.
- Servicio de análisis: encargado de generar métricas del sistema como productividad de usuarios, tareas completadas, estadísticas generales y dashboard administrativo.
- Frontend: interfaz de usuario que permite interactuar con el sistema.

Cada uno de estos servicios se ejecuta en un contenedor Docker independiente, lo que permite su aislamiento y facilita su despliegue.

La comunicación entre los servicios se realiza mediante peticiones HTTP siguiendo un estilo REST. El frontend actúa como cliente, consumiendo los servicios de usuarios y tareas, mientras que los servicios backend también se comunican entre sí para cumplir ciertas funcionalidades.

Por ejemplo, el servicio de notificaciones consulta tanto el servicio de tareas como el de usuarios para enviar recordatorios, mientras que el servicio de análisis consume información del servicio de tareas para generar métricas y reportes del sistema.

El sistema sigue un modelo cliente-servidor, donde el frontend realiza solicitudes a los microservicios, y estos procesan la información y devuelven respuestas.

Esta arquitectura permite separar responsabilidades, facilitar el mantenimiento del sistema y representar un enfoque típico de sistemas distribuidos, donde múltiples componentes independientes colaboran para ofrecer una funcionalidad completa.

Descripción del avance

En esta etapa del proyecto se logró pasar de una propuesta conceptual a una implementación funcional del sistema TaskAPI.

Se implementaron los servicios de usuarios y tareas, permitiendo el registro, autenticación y gestión de tareas dentro del sistema. Además, se integró un mecanismo de autenticación basado en JWT para proteger los endpoints y controlar el acceso según el rol del usuario.

Como parte del avance, se desarrolló el servicio de notificaciones, el cual interactúa con los demás servicios para consultar tareas pendientes y enviar recordatorios a los usuarios.

También se incorporó el servicio de análisis, encargado de generar métricas del sistema como el total de tareas, tareas completadas, productividad por usuario y estadísticas generales, permitiendo una visión más completa del comportamiento del sistema.

Además, se desarrolló una interfaz de usuario que permite consumir los servicios de forma más intuitiva.

Finalmente, se logró contenerizar cada componente utilizando Docker y se configuró Docker Compose para ejecutar todos los servicios de manera integrada, permitiendo que se comuniquen entre sí dentro de una misma red.

Funcionalidades implementadas

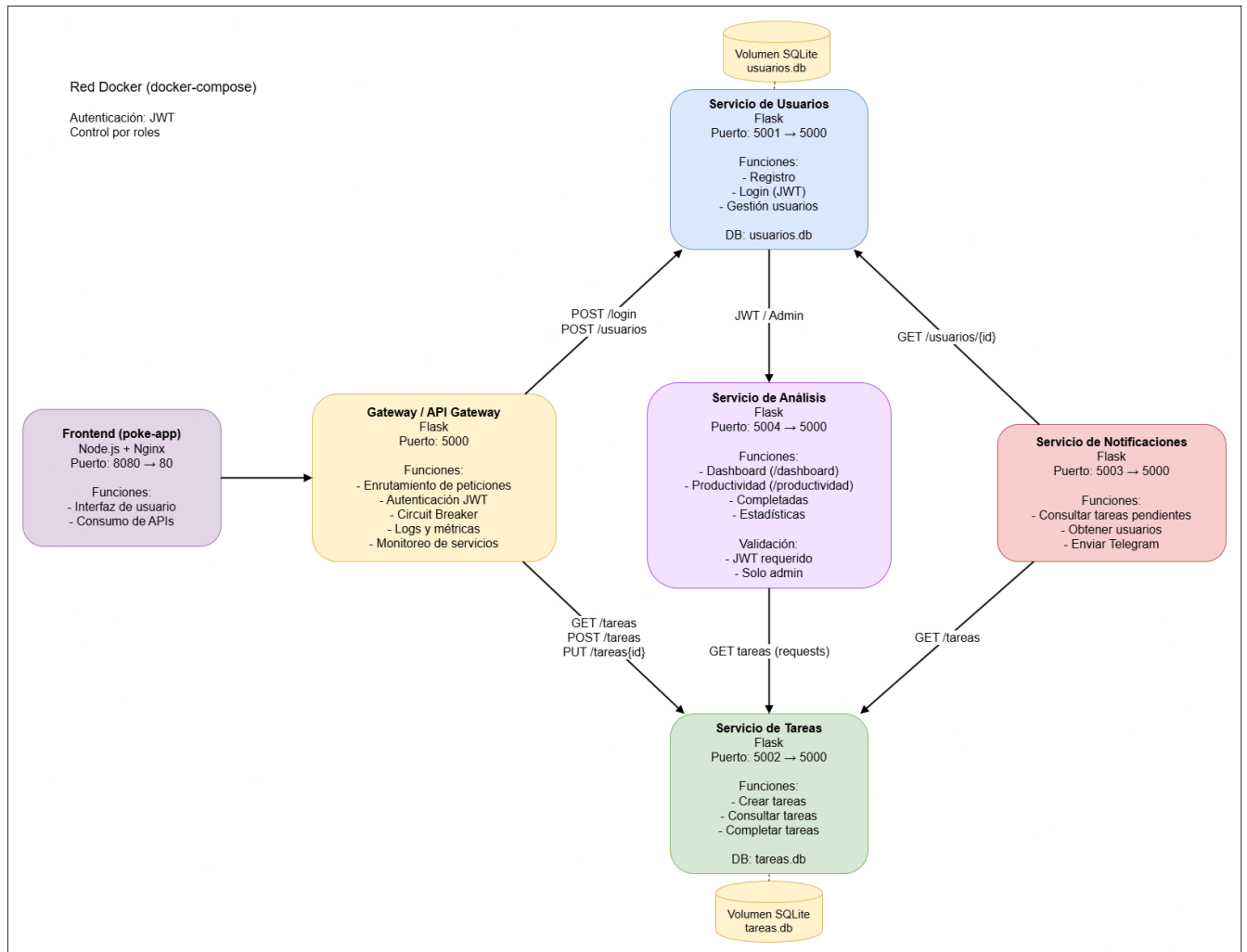
- Registro de usuarios.
- Inicio de sesión mediante autenticación con JWT.
- Creación automática de un usuario administrador al iniciar el sistema.
- Gestión de tareas:
 - o Creación de tareas (solo administrador)
 - o Consulta de tareas (según rol)
 - o Marcado de tareas como completadas
- Asociación de tareas a usuarios.
- Envío de notificaciones de tareas pendientes mediante Telegram.
- Generación de métricas y estadísticas del sistema mediante el servicio de análisis. (solo administrador)
- Comunicación entre servicios mediante API REST.
- Control de acceso basado en roles (admin / usuario).
- Contenerización completa del sistema utilizando Docker.
- Persistencia de datos mediante bases de datos SQLite.

Componentes no implementados o en mejora

- Manejo avanzado de errores entre servicios.
- Implementación de una base de datos centralizada (actualmente cada servicio usa SQLite).
- Mecanismos de alta disponibilidad o escalabilidad automática.

Arquitectura actualizada

El sistema implementa una arquitectura de microservicios donde cada componente funciona de manera independiente dentro de un contenedor Docker. Se utilizó la herramienta draw.io (diagrams.net) para la construcción del diagrama actualizado del sistema.



Servicios existentes

■ Servicio de Usuarios - Puerto: 5001 o
Tecnologías: Python, Flask, SQLite

Funciones:

- Registro de usuarios
 - Inicio de sesión (generación de JWT)
 - Gestión de roles (admin / usuario)
 - Creación automática de usuario administrador
-

■ Servicio de Tareas - Puerto: 5002 o
Tecnologías: Python, Flask, SQLite

Funciones:

- Creación de tareas (solo admin)
 - Consulta de tareas
 - Marcado de tareas como completadas
-

■ Servicio de Notificaciones - Puerto: 5003 Funciones:

- Consulta de tareas pendientes
 - Obtención de usuarios
 - Envío de notificaciones mediante Telegram
-

■ Servicio de Análisis - Puerto: 5004 Funciones:

- Generación de métricas generales del sistema
 - Dashboard administrativo (/dashboard)
 - Productividad por usuario (/productividad)
 - Conteo de tareas completadas y pendientes
 - Estadísticas generales del sistema
 - Validación de acceso mediante JWT y rol admin
-

■ Frontend (Interfaz de usuario) - Puerto: 8080 o
Tecnologías: Node.js, Nginx

Función:

- Interfaz gráfica para interacción del usuario
 - Consumo de servicios backend
-

Relación entre servicios

El sistema está compuesto por múltiples servicios que interactúan entre sí mediante solicitudes HTTP dentro de una red interna gestionada por Docker Compose.

1. Interacción del cliente (Frontend → Backend).

El frontend actúa como punto de entrada del usuario y se comunica directamente con los siguientes servicios:

- Servicio de usuarios
 - Registro de nuevos usuarios
 - Autenticación (login y obtención de JWT)
- Servicio de tareas
 - Consulta de tareas
 - Marcado de tareas como completadas

2. Comunicación entre microservicios.

Los servicios backend también se comunican entre sí para cumplir ciertas funcionalidades:

- Servicio de tareas → Servicio de usuarios
 - Validación de usuarios al asignar o consultar tareas
- Servicio de análisis → Servicio de tareas
 - Obtención de información de tareas para generar métricas y estadísticas del sistema

3. Procesos internos (servicio de notificaciones).

El servicio de notificaciones opera de forma independiente y consulta otros servicios para ejecutar su lógica:

- Notificaciones → Servicio de tareas
 - Obtiene las tareas pendientes
- Notificaciones → Servicio de usuarios
 - Obtiene la información de contacto de los usuarios (chat_id)
- Notificaciones → Servicio externo (Telegram)
 - Envía mensajes de notificación a los usuarios

4. Procesos del servicio de análisis.

El servicio de análisis se encarga de procesar información del sistema para generar métricas administrativas:

- Análisis → Servicio de tareas
 - Obtiene todas las tareas para calcular estadísticas
- Análisis → Servicio de usuarios (indirectamente vía token/roles)
 - Valida permisos de acceso a dashboards y métricas

Endpoints principales

Servicio de usuarios

POST /login → autenticación y generación de token

POST /usuarios → registro de usuario

GET /usuarios → consulta de usuarios (solo admin)

Servicio de tareas

GET /tareas → consulta de tareas

POST /tareas → creación de tareas (solo admin)

PUT /tareas/{id}/completar → marcar tarea como completada

Servicio de análisis

GET /dashboard → métricas generales del sistema

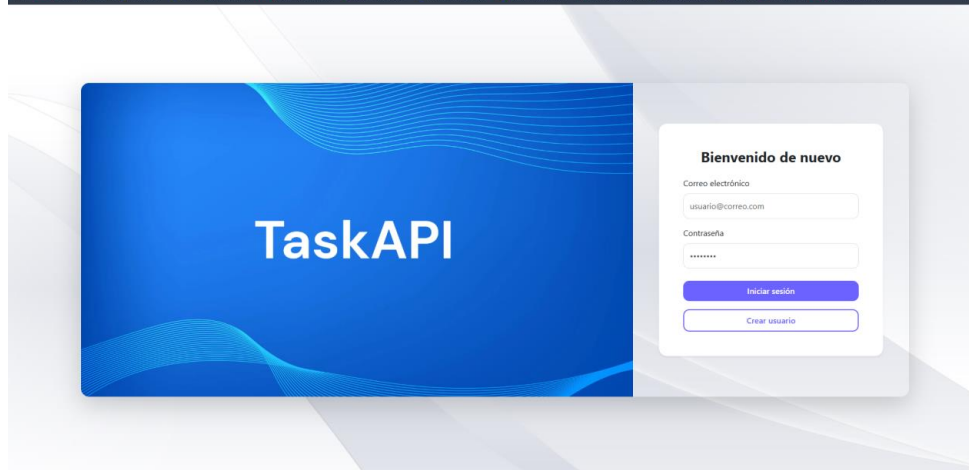
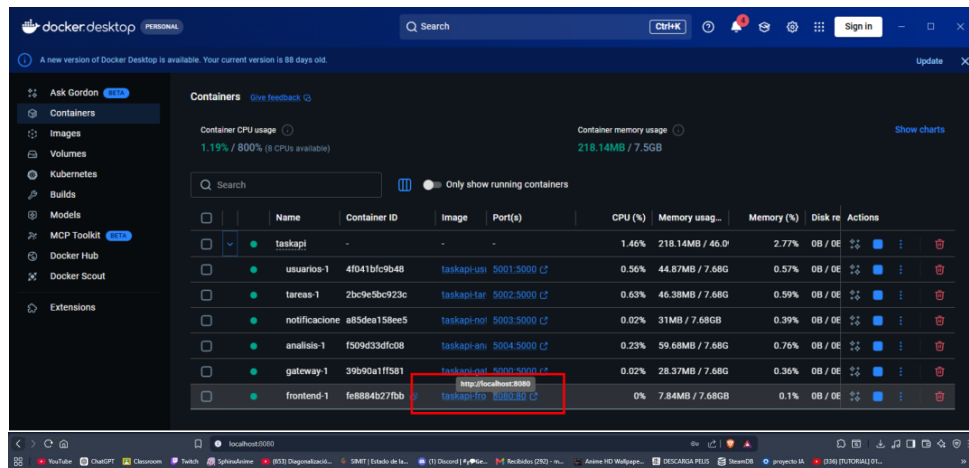
GET /productividad → productividad por usuario

GET /completadas → tareas completadas

GET /estadisticas → estadísticas generales

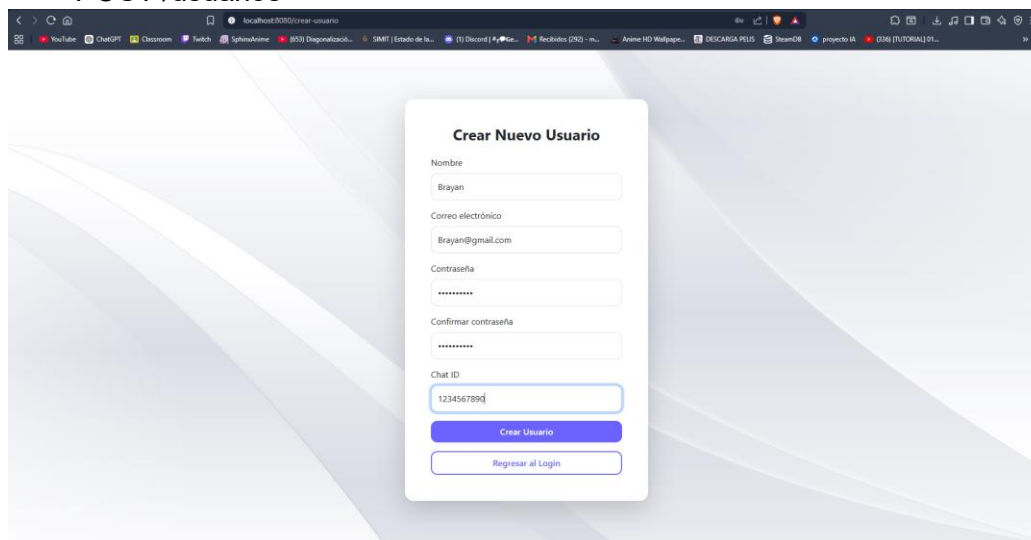
Flujo completo de interacción (ADMIN)

1. El admin accede al sistema mediante el frontend.

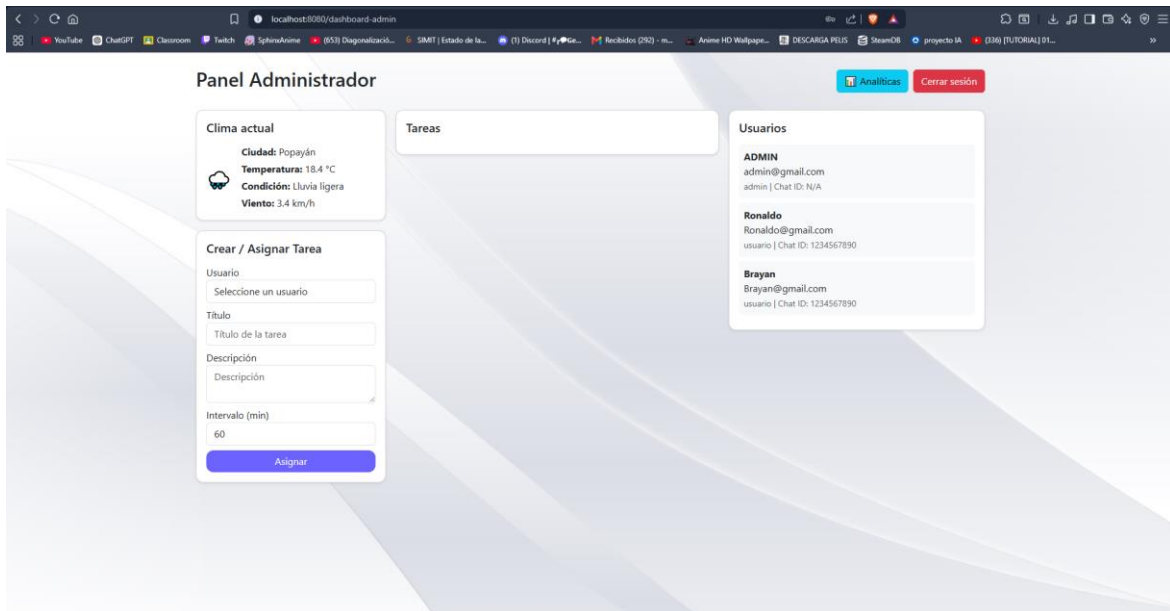


2. El admin se ingresa un nuevo usuario:

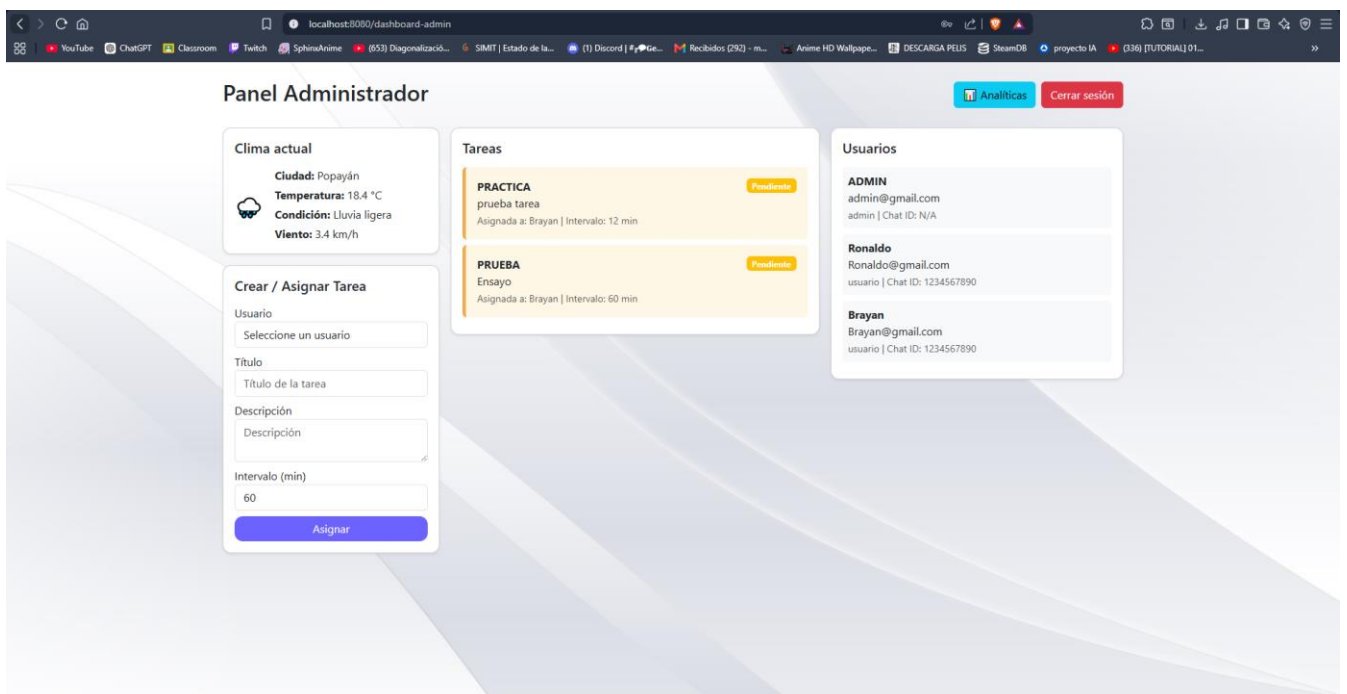
- POST /usuarios



3. El admin se autentica:
 - POST /login
 - Obtiene un token JWT.
4. El frontend utiliza el token para consumir el servicio de tareas:
 - GET /tareas

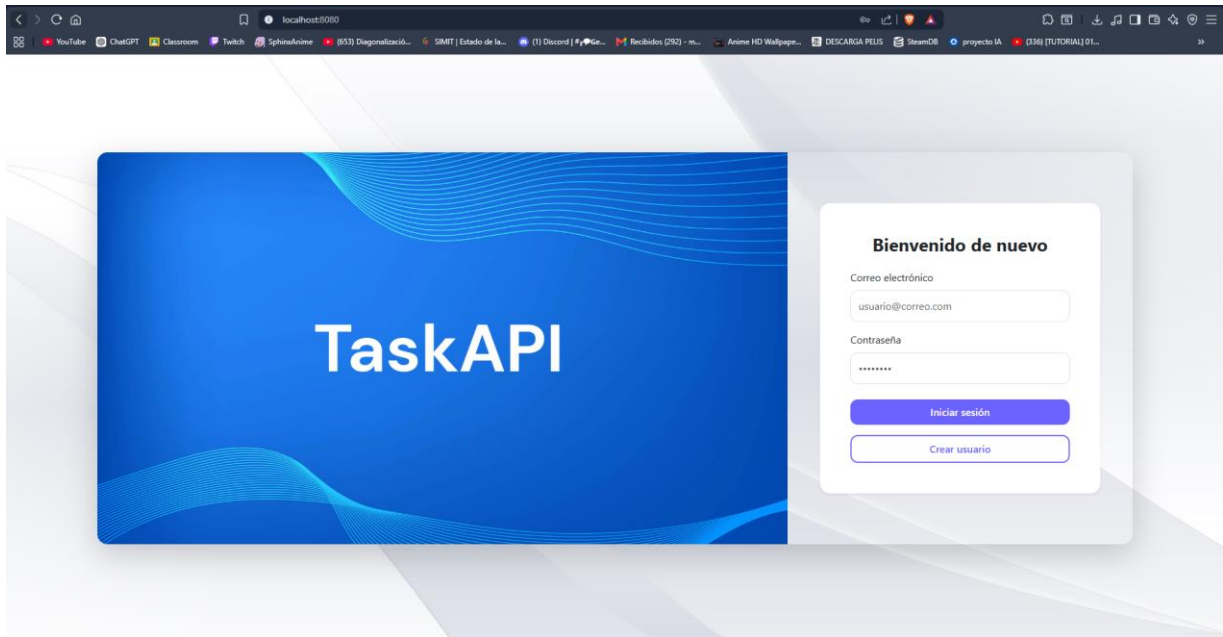


5. El administrador crea tareas:
 - a. POST /tareas



Flujo completo de interacción (USUARIO)

1. El usuario accede al sistema mediante el frontend.

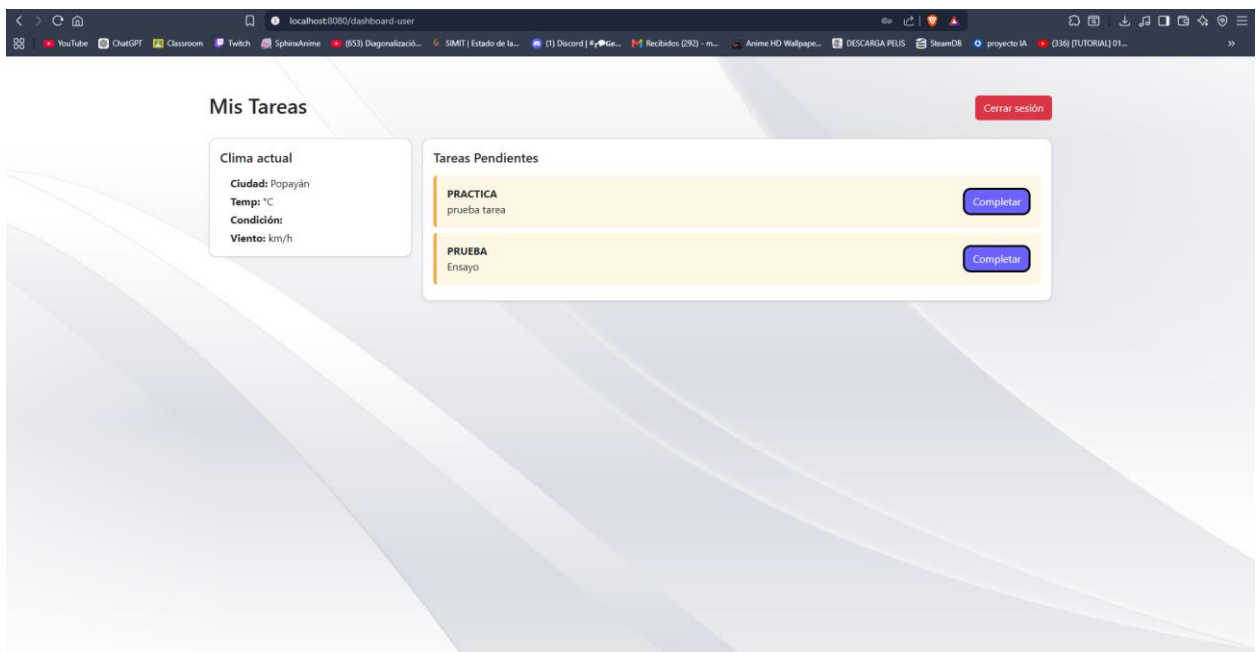


2. El usuario se autentica:

- POST /login
- Obtiene un token JWT.

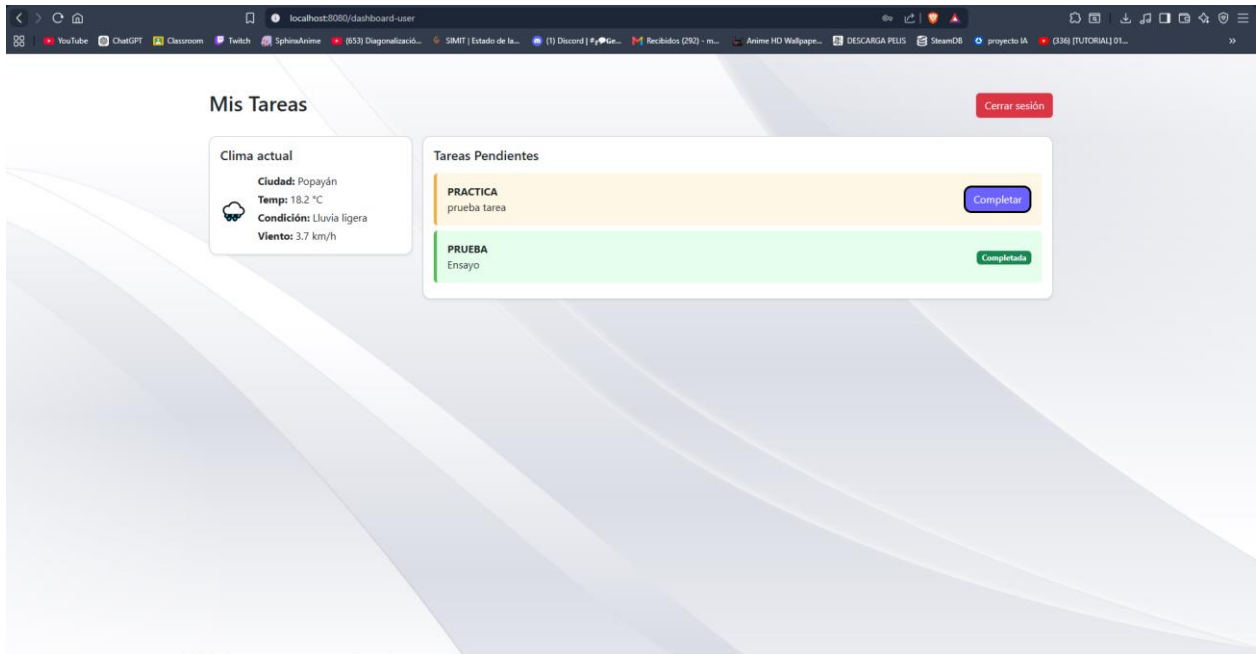
3. El frontend utiliza el token para consumir el servicio de tareas:

- GET /tareas

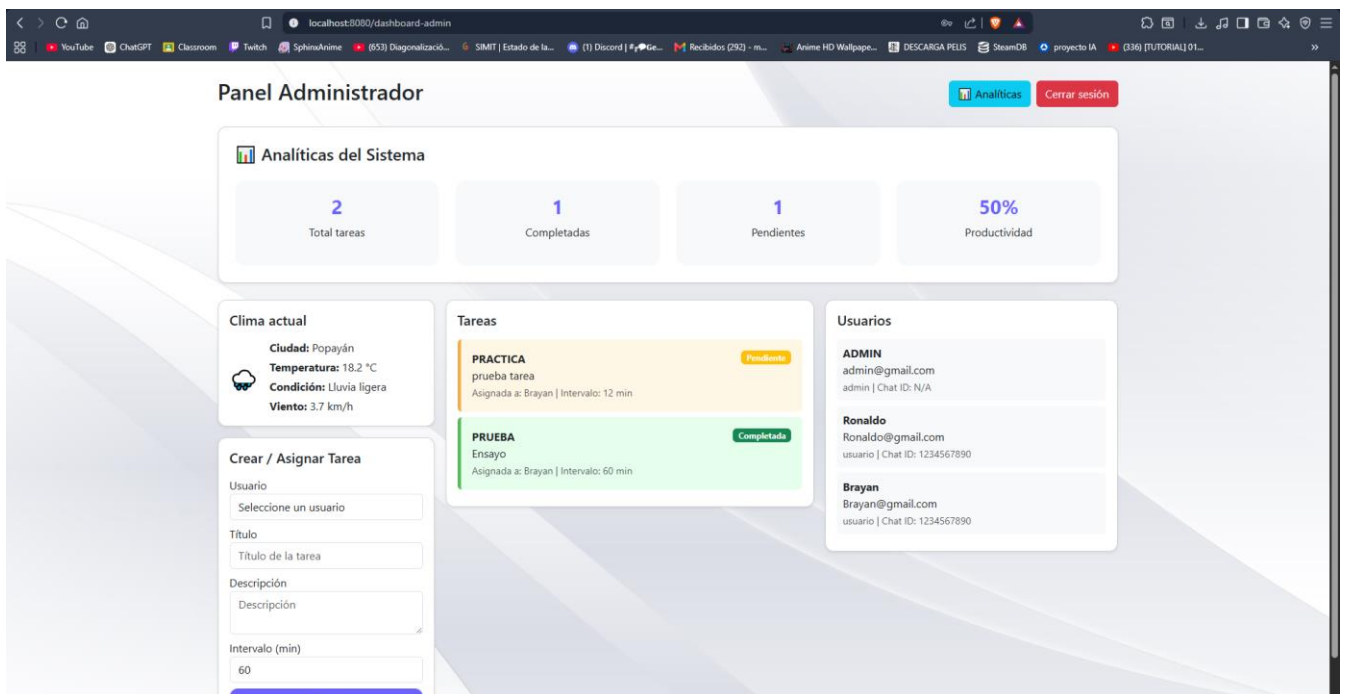


4. El usuario consulta y completa tareas:

- PUT /tareas/{id}/completar



Analíticas (ADMIN)



1. El Admin puede ver un análisis de productividad:

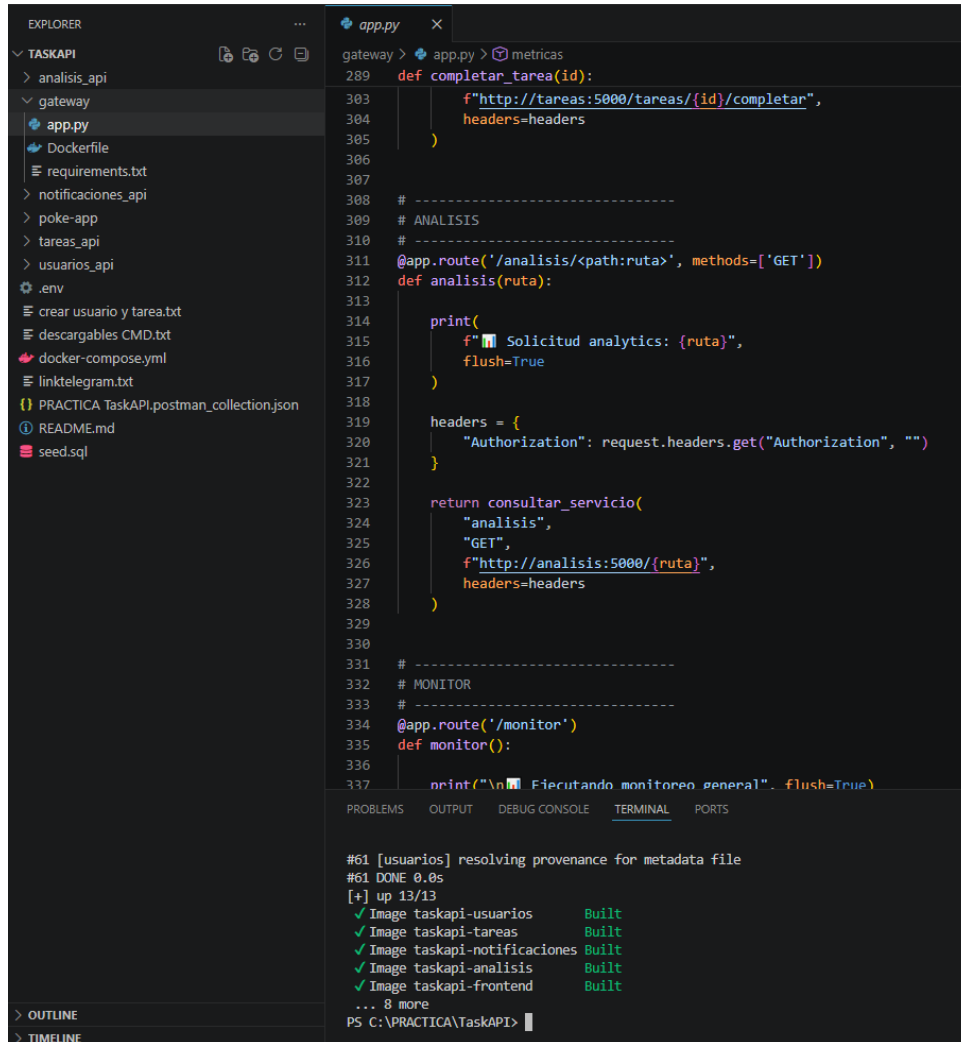
- GET /tareas/{requests}

Flujo completo de interacción (NOTIFICACIONES)

El servicio de notificaciones:

- Consulta tareas pendientes (GET /tareas)
- Obtiene usuarios (GET /usuarios)
- Envía notificaciones mediante Telegram

Prototipo Técnico en ejecución

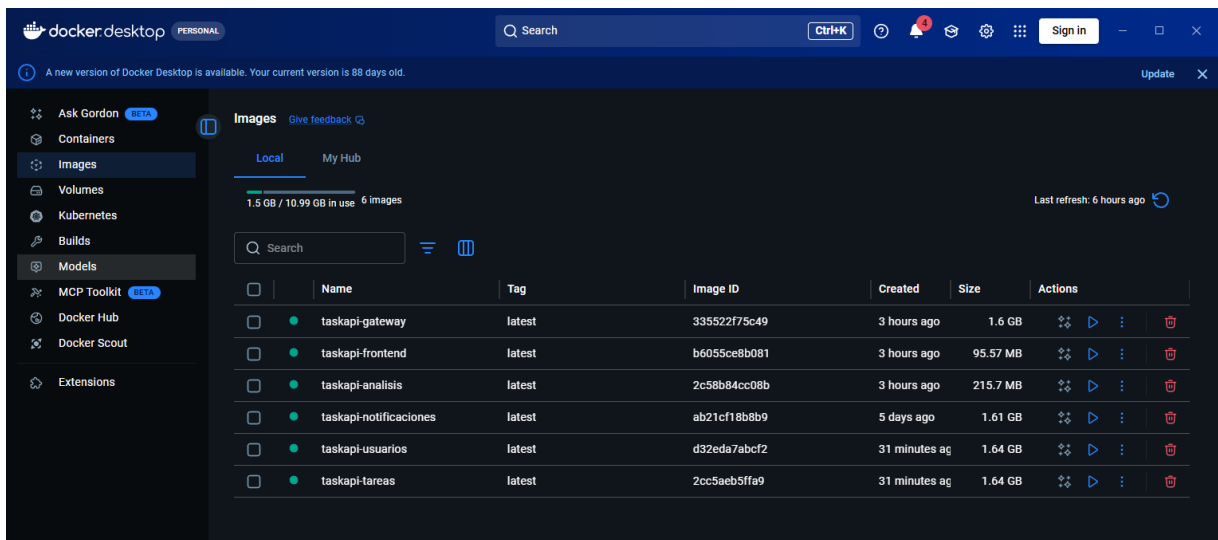


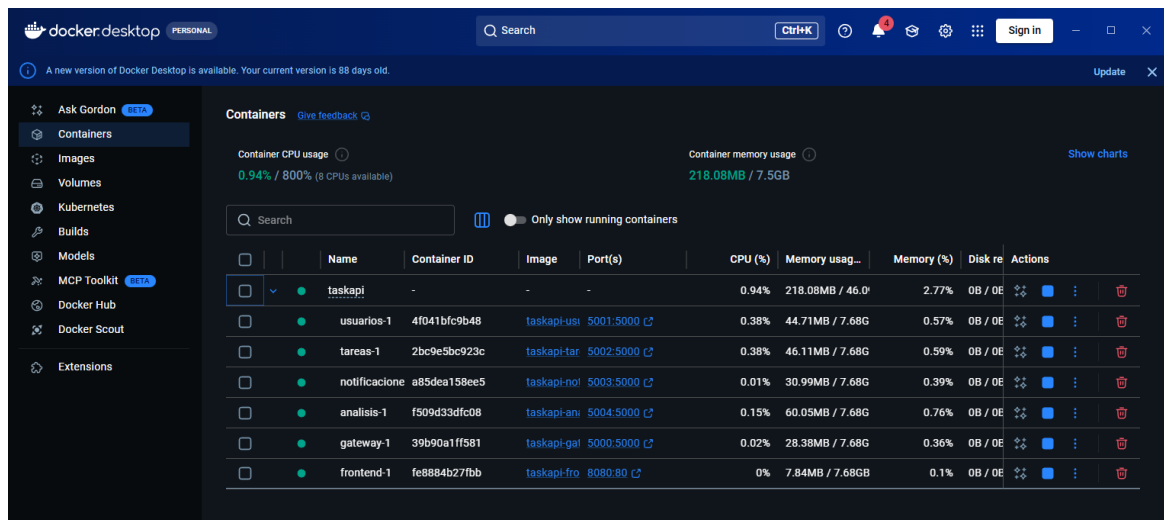
The screenshot shows a VS Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure for 'TASKAPI' with subdirectories for 'analisis_api', 'gateway', 'notificaciones_api', 'poke-app', 'tarefas_api', 'usuarios_api', and files like 'Dockerfile', 'requirements.txt', '.env', 'crear usuario y tarea.txt', 'descargables CMD.txt', 'docker-compose.yml', 'linktelegram.txt', 'PRACTICA TaskAPI.postman_collection.json', 'README.md', and 'seed.sql'. The code editor shows the 'app.py' file in the 'gateway' directory. The code defines a Flask application with routes for 'completar_tarea', 'analisis', and 'monitor'. The 'analisis' route calls a service to get analytics data. The 'monitor' route prints a message. The terminal at the bottom shows the output of the application, including a message about resolving provenance for metadata file and a list of built images: taskapi-usuarios, taskapi-tareas, taskapi-notificaciones, taskapi-analisis, and taskapi-frontend.

```
gateway > app.py > métricas
289 def completar_tarea(id):
303     f"http://tarefas:5000/tarefas/{id}/completar",
304     headers=headers
305 )
306
307
308 # -----
309 # ANALISIS
310 # -----
311 @app.route('/analisis/<path:ruta>', methods=['GET'])
312 def analisis(ruta):
313
314     print(
315         f"📄 Solicitud analytics: {ruta}",
316         flush=True
317     )
318
319     headers = {
320         "Authorization": request.headers.get("Authorization", "")
321     }
322
323     return consultar_servicio(
324         "analisis",
325         "GET",
326         f"http://analisis:5000/{ruta}",
327         headers=headers
328     )
329
330
331 # -----
332 # MONITOR
333 # -----
334 @app.route('/monitor')
335 def monitor():
336
337     print("\n📄 Ejecutando monitoreo general", flush=True)
```

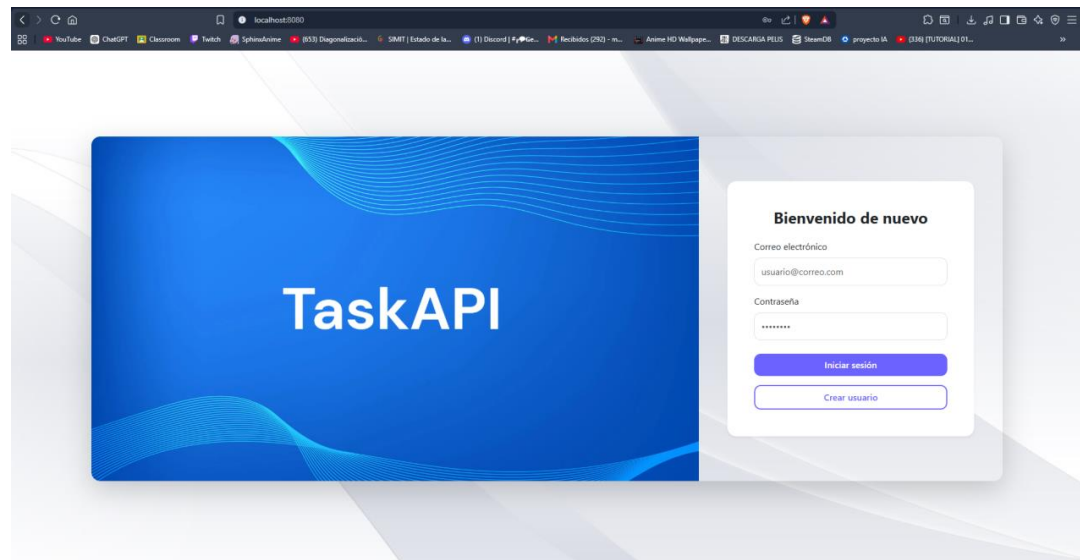
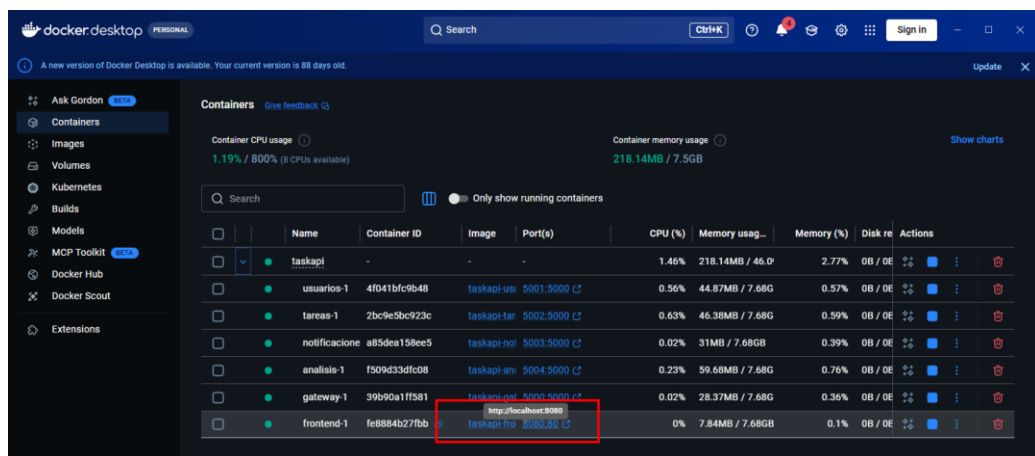
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

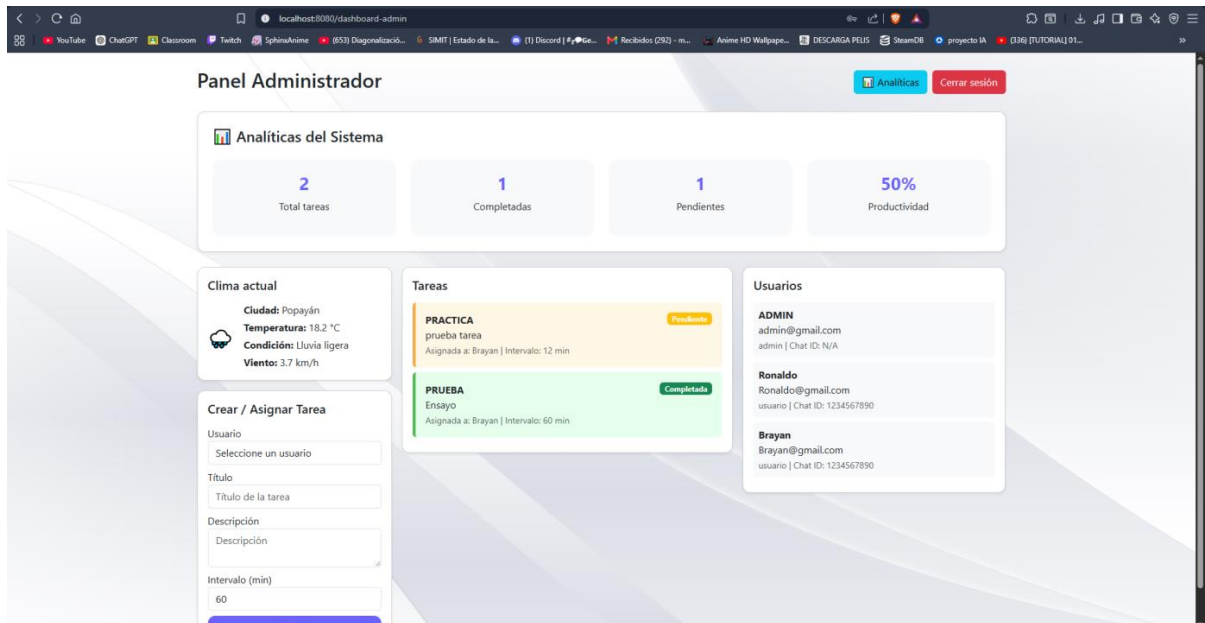
```
#61 [usuarios] resolving provenance for metadata file
#61 DONE 0.0s
[+] up 13/13
✓ Image taskapi-usuarios      Built
✓ Image taskapi-tareas        Built
✓ Image taskapi-notificaciones Built
✓ Image taskapi-analisis      Built
✓ Image taskapi-frontend      Built
... 8 more
PS C:\PRACTICA\TaskAPI>
```





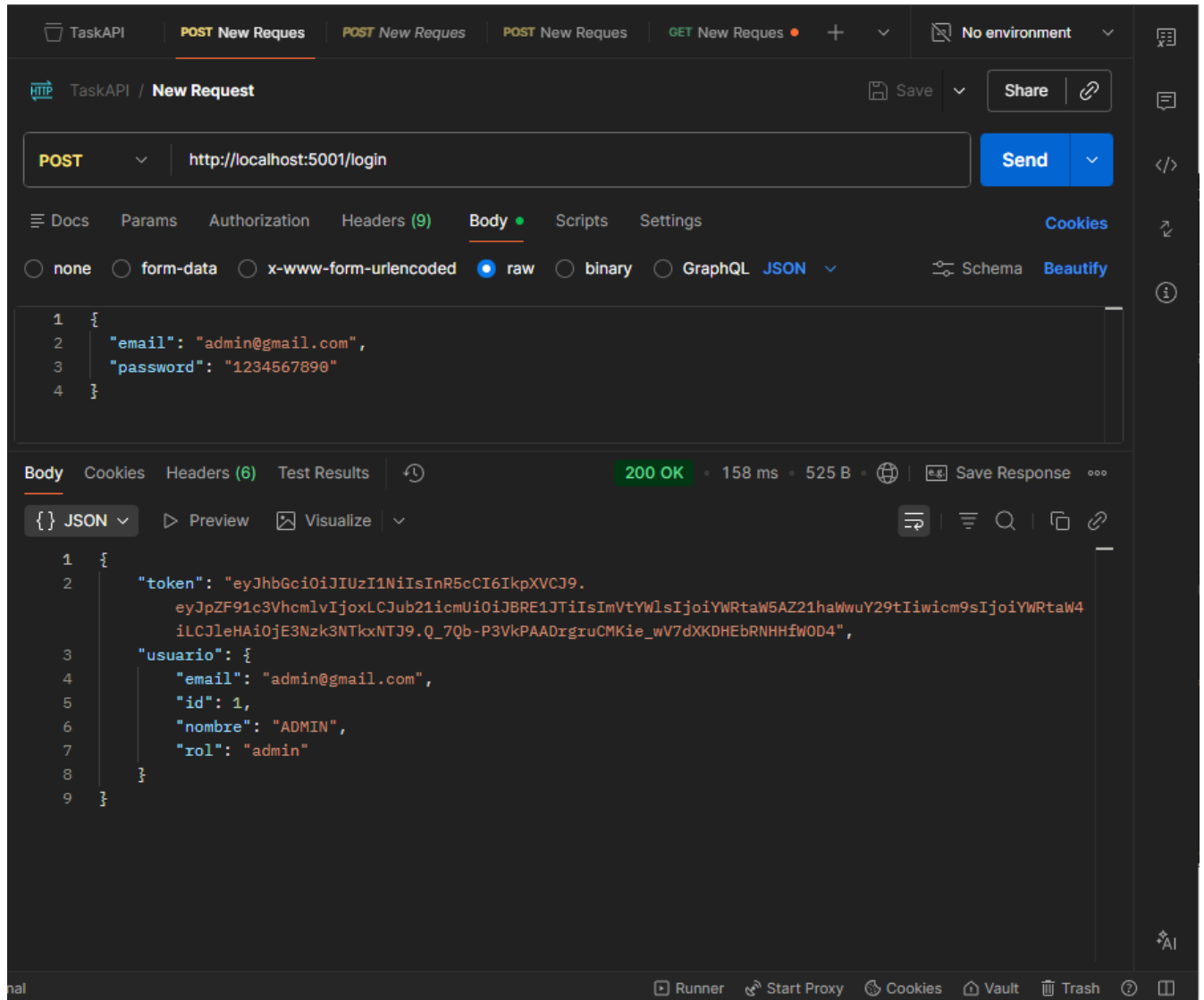
Acceso al frontend



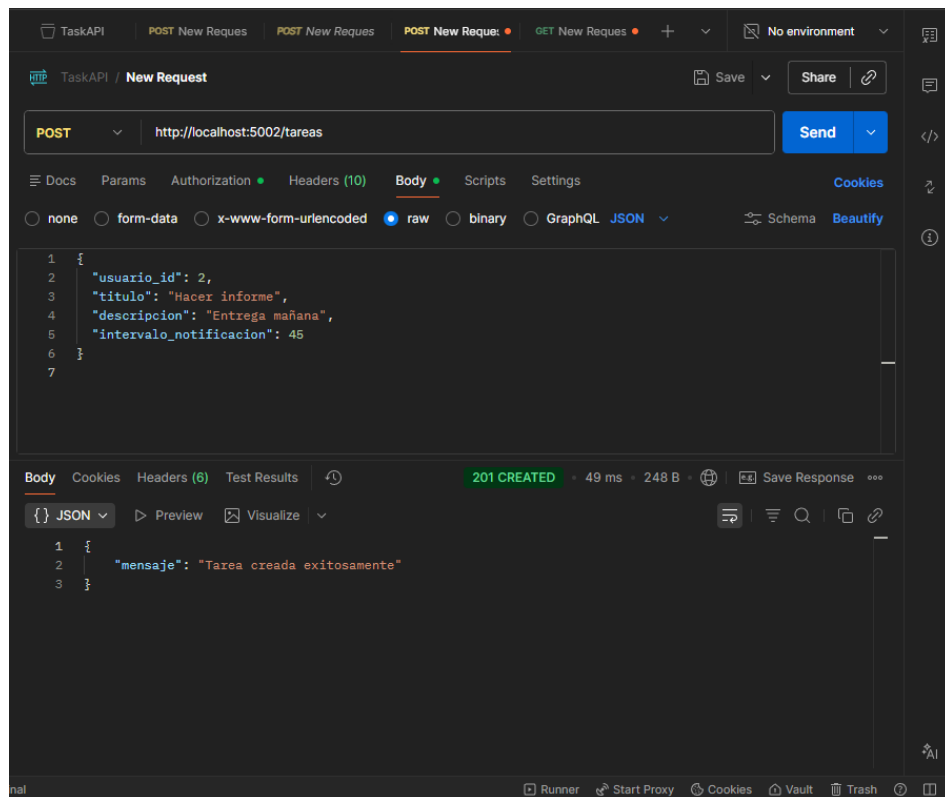
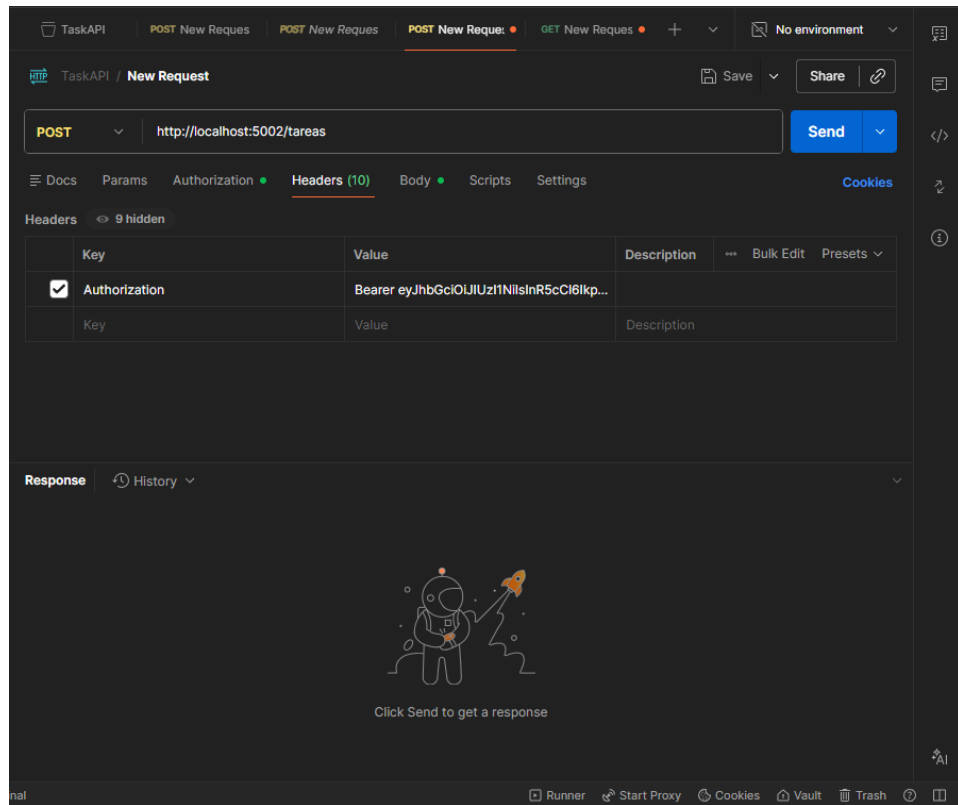


Endpoints postman

Login:



Crear tarea y asignarla a un usuario:



Ver tareas creadas:

TaskAPI x POST New Reques TheDogAPI POST New Reque: GET New Reques + No environment

TaskAPI / New Request Save Share

GET http://localhost:5002/tareas Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

Headers 7 hidden

	Key	Value	Description	Bulk Edit	Presets
☒	Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6Ikp...			
	Key	Value	Description		

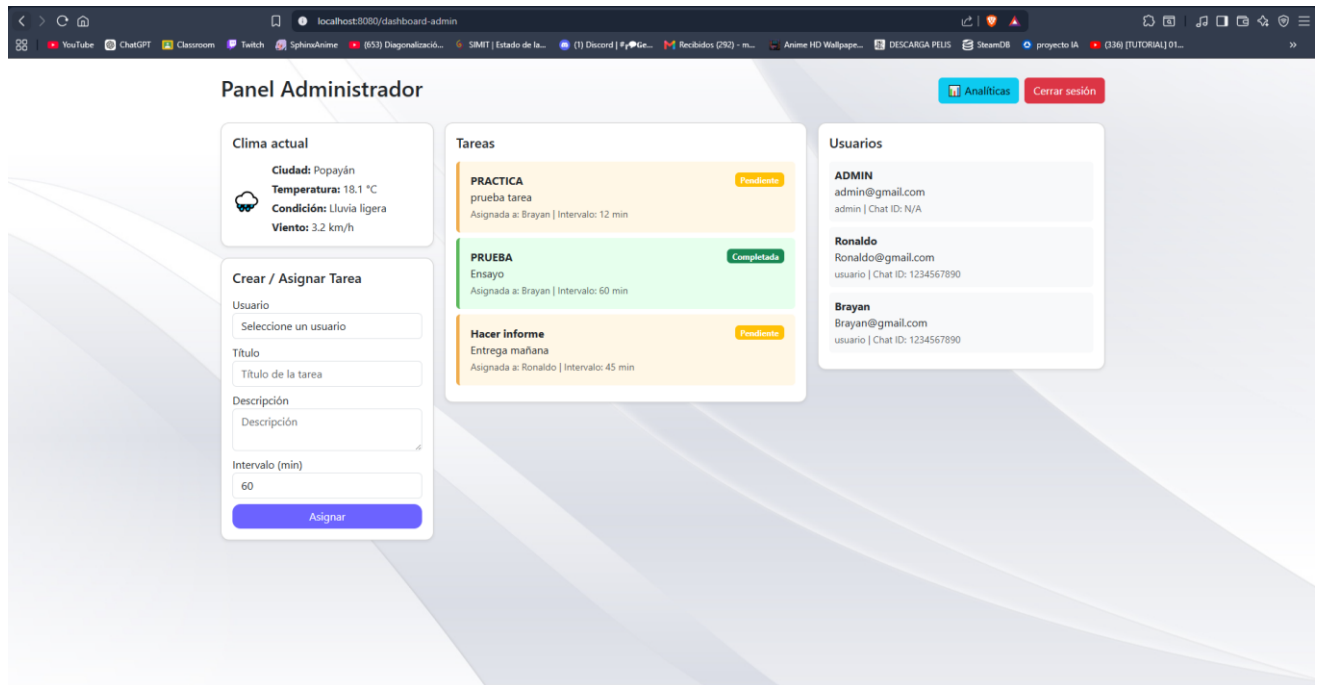
Body Cookies Headers (6) Test Results 200 OK 30 ms 757 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "tareas": [
3     {
4       "completada": false,
5       "descripcion": "prueba tarea",
6       "id": 1,
7       "intervalo_notificacion": 12,
8       "titulo": "PRACTICA",
9       "usuario_id": 3
10    },
11    {
12      "completada": true,
13      "descripcion": "Ensayo",
14      "id": 2,
15      "intervalo_notificacion": 60,
16      "titulo": "PRUEBA",
17      "usuario_id": 3
18    },
19    {
20      "completada": false,
21      "descripcion": "Entrega mañana",
22      "id": 3,
23      "intervalo_notificacion": 45,
24      "titulo": "Hacer informe",
25      "usuario_id": 2
26    }
27  ]
28 }
```

Runner Start Proxy Cookies Vault Trash ?

Vista en frontend:



Datos SQL en Docker:

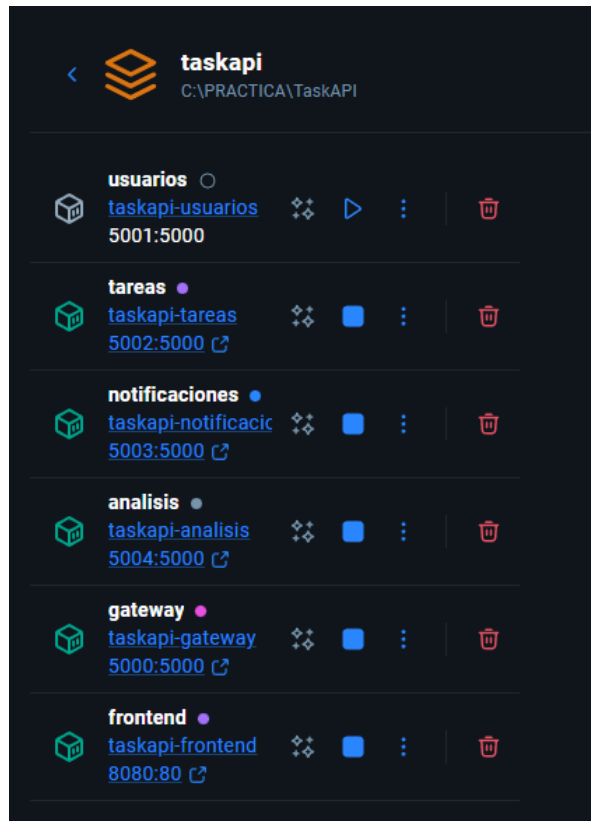
Datos almacenados en base de datos usuarios

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[+] up 13/13
✓ Image taskapi-usuarios Built 9.4s
✓ Image taskapi-tareas Built 9.4s
✓ Image taskapi-notificaciones Built 9.4s
✓ Image taskapi-analisis Built 9.4s
✓ Image taskapi-frontend Built 9.4s
... 8 more
PS C:\PRACTICA\TaskAPI> docker exec -it taskapi-usuarios-1 bash
root@4f041bfc9b48:/app# sqlite3 usuarios.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .tables
usuarios
sqlite> SELECT * FROM usuarios;
1|ADMIN|admin@gmail.com|pbkdf2:sha256:1000000$lusXscLAdjjq0fWx$38dc74a3d124d54a9e833b4893941232c75c2c3aaf4a48eabb2b95108240b9e2||admin
2|Ronaldo|Ronaldo@gmail.com|pbkdf2:sha256:1000000$0TTzRz1KWaUTDd31$be7c2534a4ac3f85ea79cd443fd448f243aa31f6c6d690c8fb5e916a5a3c9ea5|1234567890|usuario
3|Brayan|Brayan@gmail.com|pbkdf2:sha256:1000000$yX3bDxMCHmQDuWCj$e7fa7521116ad0ee428a6cfa39c94301d4432a5b0b7e0a74b8573db61033303d|1234567890|usuario
sqlite> |
```

Datos almacenados en base de datos tareas

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\PRACTICA\TaskAPI> docker exec -it taskapi-tareas-1 bash
root@2bc9e5bc923c:/app# sqlite3 tareas.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .tables
tareas
sqlite> SELECT * FROM tareas;
1|3|PRACTICA|prueba tarea|0|12
2|3|PRUEBA|Ensayo|1|60
3|2|Hacer informe|Entrega mañana|0|45
sqlite> |
```

Logs Gateway:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Solicitud recibida en /usuarios [GET]

[GATEWAY] Consultando servicio: usuarios
✅ usuarios respondió correctamente en 0.0081 segundos
172.18.0.1 - - [26/May/2026 00:42:26] "GET /usuarios HTTP/1.1" 200 -
172.18.0.1 - - [26/May/2026 00:42:26] "OPTIONS /tareas HTTP/1.1" 200 -
Solicitud recibida en /tareas [GET]

[GATEWAY] Consultando servicio: tareas
✅ tareas respondió correctamente en 0.0063 segundos
172.18.0.1 - - [26/May/2026 00:42:26] "GET /tareas HTTP/1.1" 200 -
172.18.0.1 - - [26/May/2026 00:42:26] "OPTIONS /analisis/dashboard HTTP/1.1" 200 -
Solicitud analytics: dashboard

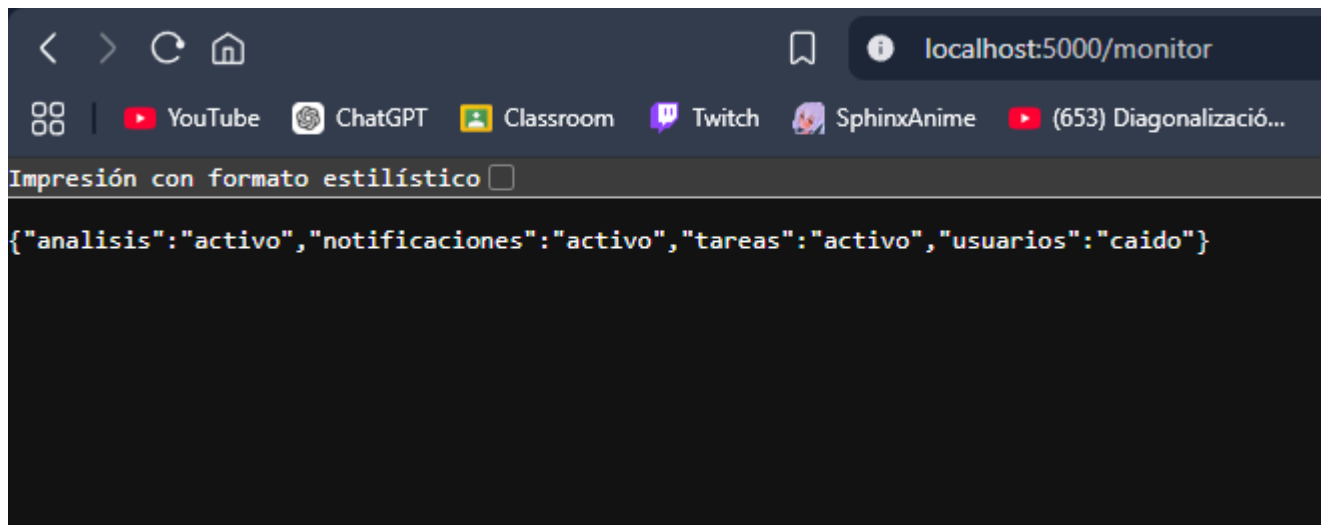
[GATEWAY] Consultando servicio: analisis
✅ analisis respondió correctamente en 0.0226 segundos
172.18.0.1 - - [26/May/2026 00:42:26] "GET /analisis/dashboard HTTP/1.1" 200 -
172.18.0.1 - - [26/May/2026 01:15:04] "OPTIONS /usuarios HTTP/1.1" 200 -
Solicitud recibida en /usuarios [GET]

[GATEWAY] Consultando servicio: usuarios
❌ ERROR conectando con usuarios
Cantidad de errores: 1
Fallos consecutivos: 1
Detalle: HTTPConnectionPool(host='usuarios', port=5000): Max retries exceeded with url: /usuarios (Caused by NameResolutionError("HTTPConnection(host='usuarios', port=5000): Failed to resolve 'usuarios' ([Errno -2] Name or service not known)"))
172.18.0.1 - - [26/May/2026 01:15:07] "GET /usuarios HTTP/1.1" 503 -
172.18.0.1 - - [26/May/2026 01:15:07] "OPTIONS /tareas HTTP/1.1" 200 -
Solicitud recibida en /tareas [GET]

[GATEWAY] Consultando servicio: tareas
✅ tareas respondió correctamente en 0.0041 segundos
172.18.0.1 - - [26/May/2026 01:15:07] "GET /tareas HTTP/1.1" 401 -
172.18.0.1 - - [26/May/2026 01:15:08] "OPTIONS /analisis/dashboard HTTP/1.1" 200 -
Solicitud analytics: dashboard

[GATEWAY] Consultando servicio: analisis
✅ analisis respondió correctamente en 0.0033 segundos
172.18.0.1 - - [26/May/2026 01:15:08] "GET /analisis/dashboard HTTP/1.1" 401 -
PS C:\PRACTICA\TaskAPI>
```

http://localhost:5000/monitor:



Circuit breaker:

